INVENTORY ROBOT

This project combines week 2 (Nav2) and week 3 (ML)

Project overview:

The **Autonomous Inventory Robot** is a fully integrated hardware-software system designed for intelligent navigation and automated visual inspection within structured environments like warehouses and industrial facilities.

The system leverages **SLAM** (Simultaneous Localization and Mapping) to generate an initial map of the operational space. Given the coordinate data for specific rooms or zones, the robot utilizes the ROS 2 **Nav2** stack and the **Simple Commander API** to execute autonomous, collision-free navigation to target locations.

Upon reaching a designated waypoint, the robot initiates its computer vision pipeline. It captures a high-resolution image of the environment and processes it through a custom-trained Convolutional Neural Network (**CNN**). The vision system utilizes a hierarchical detection strategy:

1. **Target Identification & Counting:** The CNN first scans the image to identify and count the presence of "red objects" (the primary regions of interest).
2. **Shape Classification:** For every red object detected, the system draws a bounding box and runs a secondary classification to determine the object's specific geometry, categorizing it strictly as a **cube**, **cylinder**, or **sphere**.

How to run:

Extract the **task4_submission.zip** file into your workspace.

RECOMMENDED: NAME THE WORKSPACE task4_ws and keep the extracted src folder in it. *(otherwise you will have to edit workspace name in commander.py file and the full_simulation.launch.py file)*

Step 1: Navigate to your workspace and build the package.

```shell
cd ~/task4_ws
colcon build --packages-select inventory_bot
```

DIRECTORY STRUCTURE:

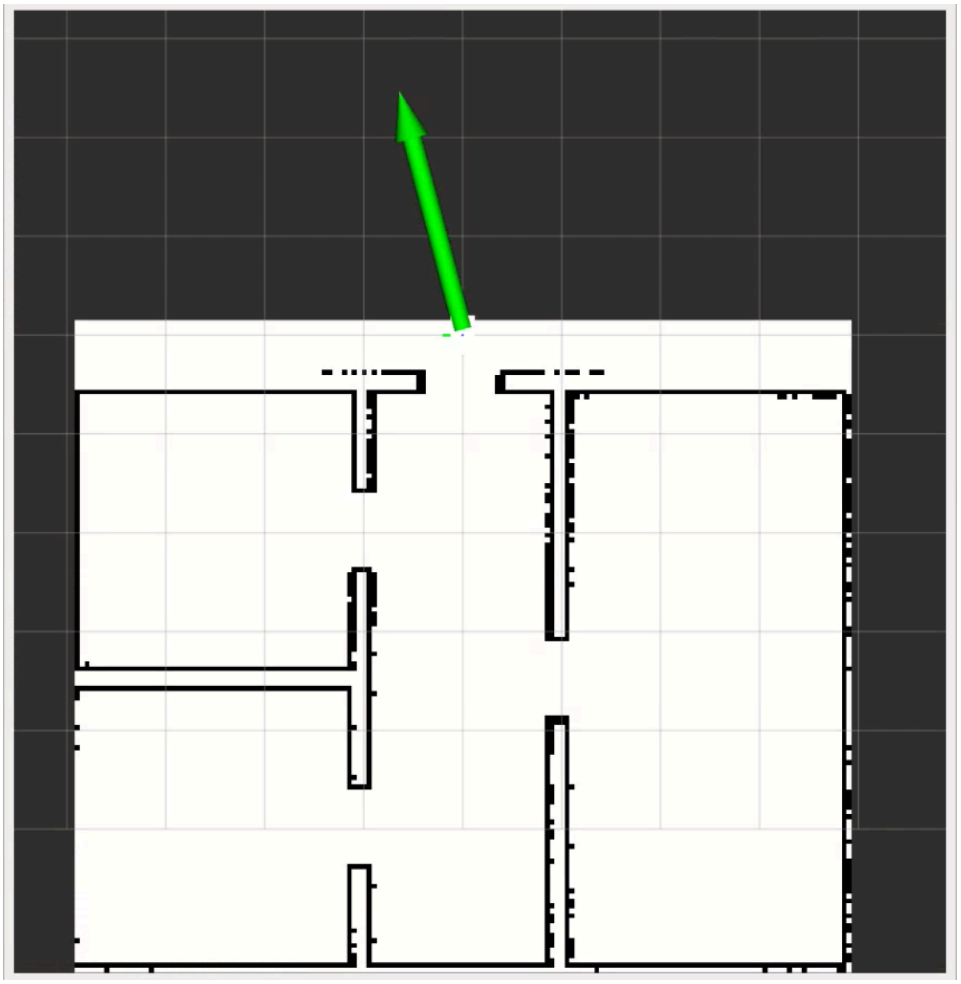
```
task4_ws/
└── src/
    └── inventory_bot/
        ├── package.xml
        ├── setup.py
        ├── setup.cfg
        ├── inventory_bot/
        │   ├── __init__.py
        │   └── commander.py      # Control and inference logic
        ├── launch/
        │   └── full_simulation.launch.py #The .launch.py file
        ├── maps/
        │   ├── warehouse.yaml    # Nav2 map yaml
        │   └── warehouse.pgm     # Map image
        ├── my_cnn/               # Vision model & metrics
        │   ├── best.onnx         # Exported model for inference
        │   ├── results.csv       # Training metrics
        │   ├── results.png       # Training charts
        │   └── training_dataset  # Dataset used during training
        ├── urdf/
        │   ├── differential_robot.urdf.xacro  #Bot description
        │   └── differential_robot.urdf
        └── worlds/
            └── warehouse.sdf     # The warehouse world
```

Step 2: Split the terminal into three panes:

Terminal 1:

```
Shell
source install/setup.bash
ros2 launch inventory_bot full_simulation.launch.py
```

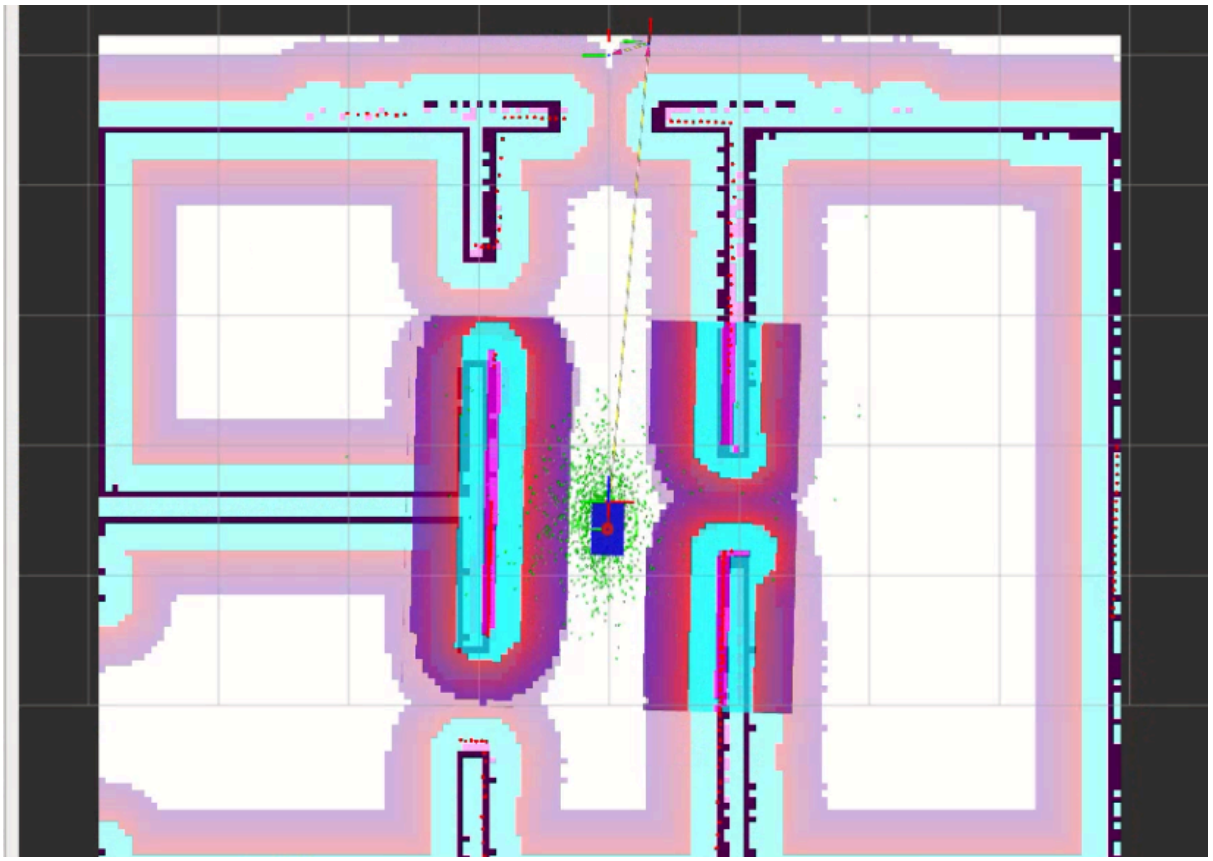
When RVIZ opens provide initial pose estimate in the forward direction:



Terminal 2:
Teleop keyboard:

```
Shell
source install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

Press the ‘ , ‘ key to drive the robot until the AMCL cloud localizes:



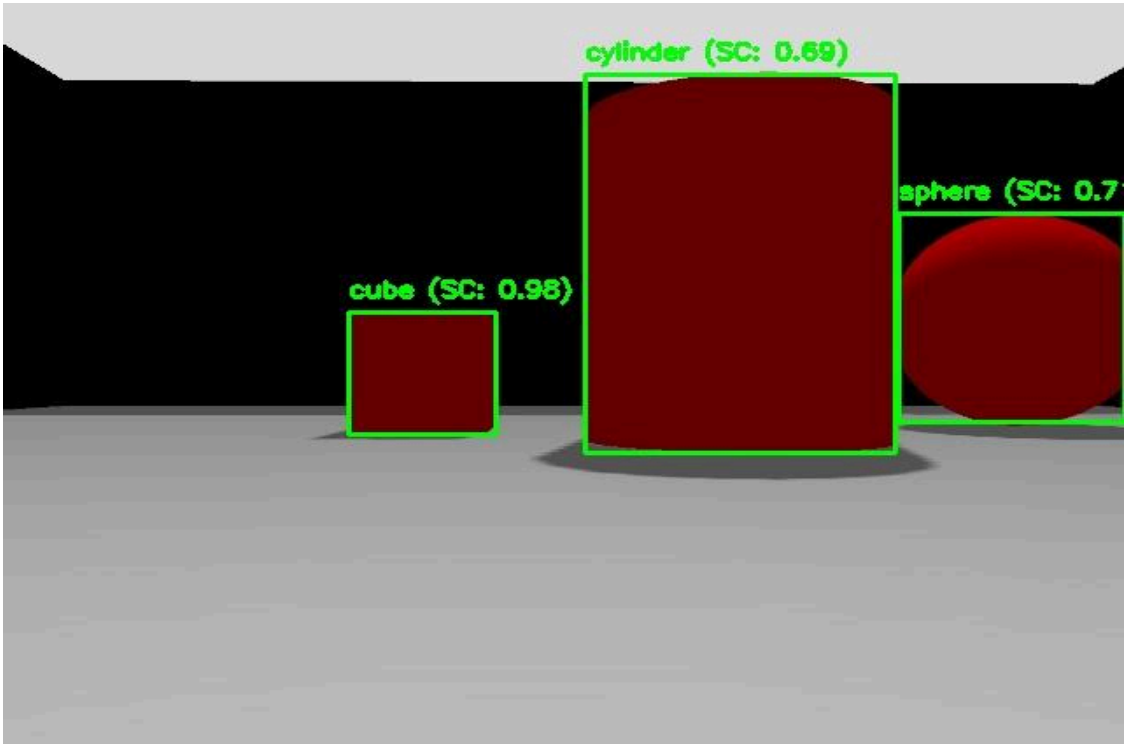
Now you may close the Teleop keyboard terminal!

Terminal 3:
Run the code for navigation and inference:

```shell
source install/setup.bash
ros2 run inventory_bot commander
```

Step 3: Observe:

The robot will navigate to every room one by one. Each time it reaches a room it clicks a photo and saves it to `src/inventory_bot/my_cnn`. The inference logic then outputs a prediction about the object it sees and prints it in terminal 3.
The predicted images with the bounding boxes are saved to `src/inventory_bot/my_cnn.`



ABOUT THE CNN MODEL:

Overview:

The robot's object detection module is powered by a lightweight, custom fine-tuned YOLOv8 object detection model. The vision system utilizes a hierarchical detection strategy. It first scans the image to identify and count the presence of `red object` instances. For every red object detected, the system draws a bounding box and runs a secondary classification to determine the object's specific geometry, categorizing it strictly as a `cube`, `cylinder`, or `sphere`.

Classes:

While the primary objective of the robot's vision system is to detect four core elements (`cube`, `cylinder`, `sphere`, and `red object`), the YOLOv8 model was intentionally trained to recognize a total of eight classes. The four additional classes are environmental boundaries: `black wall`, `blue wall`, `green wall`, and `yellow wall`.

Including these structural background classes drastically improved the model's performance and stability for the following reasons:

- **Prevents confusion:** Neural networks will often "hallucinate" objects in empty spaces if a shadow, reflection, or texture vaguely resembles a target class. By explicitly labeling the walls, the network doesn't have to guess what that background texture as it has a dedicated category for it. This prevents the model from mistakenly classifying a weird shadow on a blue wall as a `sphere`.
- **Allows Color Discrimination:** Because the primary target relies heavily on color (`red object`), feeding the network massive, labeled expanses of opposing colors (`blue wall`, `green wall`, `yellow wall`) forces the convolutional layers to develop highly aggressive color-filtering weights. It teaches the model exactly what *isn't* the target, making the detection of red objects significantly more robust under varying warehouse lighting.

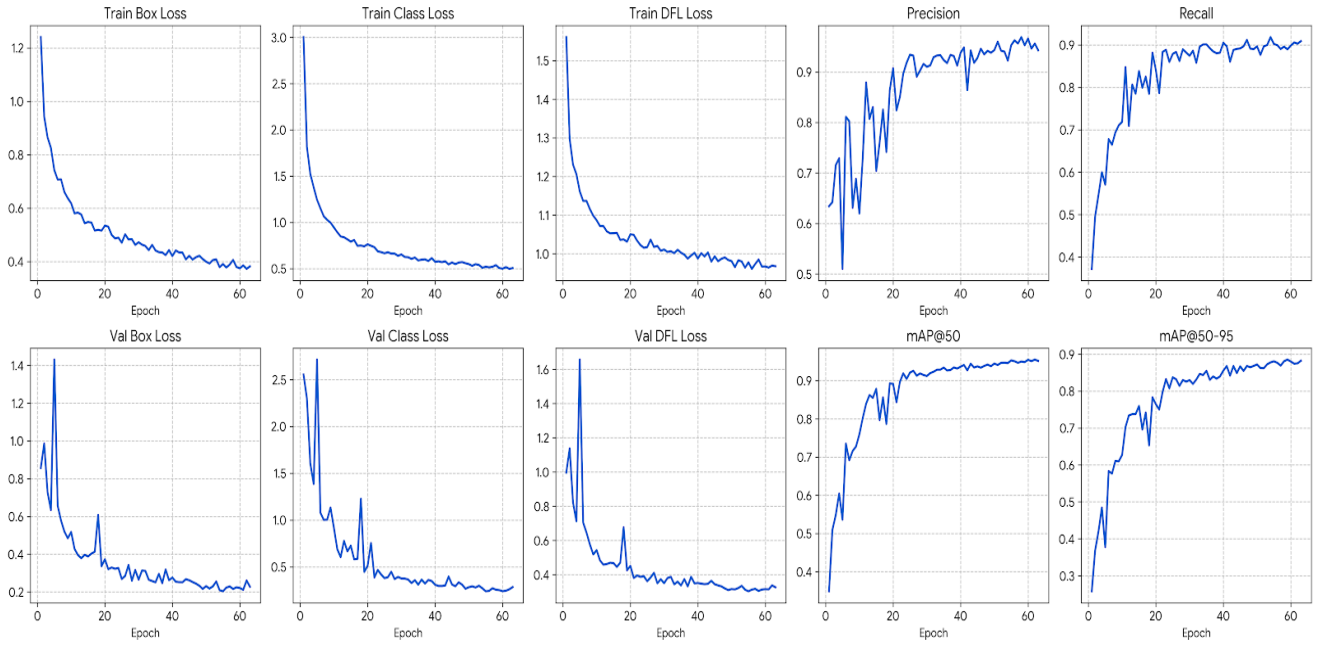
Training statistics:

The model was trained on a highly curated dataset of exactly **1,306 images** (located in `src/inventory_bot/my_cnn/training_dataset`). The dataset was constructed using a hybrid approach of custom simulation data and established machine learning datasets:

1. **Simulation Capture:** 102 images were manually captured by capturing screenshots in a dedicated Gazebo data-collection environment (`src/inventory_bot/worlds/perception_training.sdf`). This ensured the model learned the exact lighting, camera angles, and textures it would face during deployment.
2. **Annotation & Augmentation:** These 102 images were hand-annotated using Roboflow. Data augmentation techniques (such as rotations, scaling, and exposure adjustments) were applied, expanding the core custom dataset to 306 images to prevent overfitting.

3. **Foundational Geometry Integration:** To ensure the neural network had a robust, generalized understanding of 3D primitives regardless of angle or lighting, an additional 1,000 images were integrated from the official TensorFlow `Shapes3D` dataset. Find here: [Shapes3D dataset].

The YOLOv8 model was fine-tuned locally on a CPU. The training process ran for 63 epochs over a span of 13 hours. Despite the hardware limitations, this meticulously balanced dataset allowed the model to converge smoothly, resulting in a highly accurate, lightweight ONNX-exported inference model ready for real-time robotic deployment.



INDUSTRIAL APPLICATIONS:

Extreme Environment Agriculture

The architecture of this robot is highly adaptable to industries requiring automated monitoring in environments hostile to human workers.

Consider an advanced indoor agricultural facility cultivating a specialized strain of mushrooms that require extreme sub-zero temperatures (e.g., -100°C) to thrive. Human entry into these growth chambers is dangerous, requiring expensive thermal gear and strict time limits, which makes manual yield inspection highly inefficient.

By retraining the robot's CNN specifically on the visual characteristics of these mushrooms, the robot can be deployed as an automated agricultural inspector. The workflow operates as follows:

- **Autonomous Patrol:** The robot navigates the freezing growth chambers without human intervention.
- **Yield Tracking:** Using its vision system, it scans the growing beds, detecting and counting the number of mature mushrooms.
- **Harvest Trigger:** Once the detected count of fully grown mushrooms matches the expected baseline for that batch, the robot transmits a signal to the facility control system, indicating that the crop is ready for automated or specialized human harvesting.

This implementation eliminates human risk, provides highly accurate real-time yield data, and optimizes the facility's harvesting schedule.

THANK YOU!

Email: 1) nightwingklol@gmail.com 2) shourya.sharma@iitg.ac.in